MOTOR HPR

Hybrid Predictive Reasoning

HOJA DE RUTA TÉCNICA · FASE ALFA

Integración del selector de triggers, consolidación del razonamiento estratificado y criterios de validación

Objetivo central
Hacer que la interacción con el motor sea sencilla y trazable, mientras la complejidad, las validaciones y las decisiones de seguridad permanecen dentro del backend.

Contexto y decisión arquitectónica

HPR vuelve formalmente a fase alfa para alinear su identidad con el rumbo técnico actual: **Hybrid Predictive Reasoning**. La prioridad pasa a ser demostrar coherencia, reproducibilidad y control del razonamiento antes de ampliar capacidades o declarar una nueva etapa.

Esta hoja de ruta toma como punto de partida la integración declarada de N1 (Búsqueda Cognitiva), N2 (Grafo de Consistencia) y N3 (Síntesis Causal), junto con las suites de pruebas reportadas como satisfactorias. Los resultados deberán conservarse como evidencia verificable en cada ejecución.

Principio rector: la interfaz facilita el uso; nunca sustituye ni evita los controles de seguridad del motor.

1. Prioridades de implementación

P0 — Línea base alfa y documentación

• Registrar commit de referencia, rama, dependencias y configuración reproducible.
• Documentar el cambio de nomenclatura en README, interfaz, documentación técnica y mensajes de ayuda.
• Definir explícitamente qué capacidades están implementadas, cuáles son experimentales y cuáles siguen siendo hipótesis.
• Conservar los resultados de las suites como artefactos de prueba con fecha, comando ejecutado y resumen.

Criterio de aceptación: otra sesión puede identificar el estado exacto del proyecto y repetir las pruebas siguiendo instrucciones documentadas.

P1 — Catálogo único de triggers

• Inventariar los triggers realmente soportados por el backend; no asumir que un comando existe solo porque aparece en la UI.
• Para cada trigger, definir identificador, descripción, parámetros requeridos, permisos, confirmación necesaria y resultado esperado.
• Marcar triggers experimentales o restringidos y excluir los que no estén disponibles.
• Mantener el catálogo sincronizado con el backend mediante una fuente única de configuración o un mecanismo explícito de validación.

Criterio de aceptación: cada opción visible corresponde a una función admitida, documentada y autorizada por el motor.

P2 — Selector desplegable junto a la barra de chat

• Añadir un menú compacto, accesible desde la zona de entrada del chat.
• Al seleccionar una opción, preparar el trigger en el campo de entrada para que la persona pueda revisar y completar los parámetros.
• Permitir tanto selección como escritura manual; evitar que el menú sea un requisito para utilizar el chat.
• Ofrecer nombres legibles y descripciones breves, con estados claros para opciones disponibles, experimentales o deshabilitadas.
• Evitar el envío automático por el mero hecho de seleccionar una opción: el usuario confirma la activación.

Criterio de aceptación: selección, edición, cancelación y envío funcionan sin bloquear la entrada normal ni duplicar comandos.

P3 — Integración segura con el pipeline

• Enrutar los triggers seleccionados y los escritos manualmente por el mismo pipeline existente.
• Mantener activas las validaciones de Nexus Root, Sovereign Gate y las políticas de confianza aplicables.
• Validar parámetros, permisos y formato en el backend; la UI no debe ser la autoridad de seguridad.
• Rechazar de forma controlada triggers desconocidos, parámetros inválidos y solicitudes no autorizadas.
• No convertir un trigger de búsqueda externa en permiso automático para confiar o ingerir contenido externo.

Criterio de aceptación: la ruta de UI no ofrece privilegios superiores a la ruta manual y no permite saltarse las barreras de seguridad.

2. Validación del razonamiento estratificado

N1 — Búsqueda Cognitiva

• Verificar la pertinencia del scoring semántico por embeddings con casos positivos, negativos y ambiguos.
• Registrar evidencia recuperada, puntuaciones y umbrales relevantes cuando corresponda.
• Probar entradas vacías, consultas fuera de alcance y resultados insuficientes.

N2 — Grafo de Consistencia

• Preparar casos con versiones compatibles, versiones en conflicto y datos incompletos.
• Comprobar que los conflictos se detectan y se explican, en lugar de resolverse silenciosamente.
• Definir el comportamiento cuando no exista evidencia suficiente para determinar cuál versión prevalece.

N3 — Síntesis Causal

• Probar contraste, evolución y comparativa mediante las plantillas lógicas existentes.
• Verificar que las conclusiones distingan hechos, inferencias y ausencia de evidencia.
• Comprobar que la síntesis no invente causas ni afirme más de lo que sostienen las entradas.

Pruebas de extremo a extremo

• Ejecutar las suites existentes: *deep_inference*, *reasoning_beta*, *iterative_planner* y *causal_synthesis*.
• Repetir al menos cinco escenarios manuales representativos, registrando entrada, trigger, salida esperada, salida observada y resultado.
• Añadir pruebas de regresión para la UI, el detector multilingüe y las reglas de seguridad que puedan verse afectadas.
• Separar fallos funcionales, fallos de seguridad y diferencias no deterministas.

3. Métricas y evidencias

Dimensión	Evidencia mínima	Criterio de salida
Pruebas	Comando, versión, total de casos, aprobados y fallidos.	Todas las suites críticas pasan; los fallos restantes quedan explicados y priorizados.
Determinismo	Repeticiones con entradas equivalentes y comparación de resultados.	Variaciones identificadas y aceptables; ninguna contradicción crítica sin explicación.
Seguridad	Casos autorizados, no autorizados, triggers desconocidos y entradas malformadas.	No hay bypass de permisos ni de controles del backend.
UI	Pruebas de selección, edición, envío, cancelación y uso manual.	No bloquea la barra de chat ni genera envíos accidentales.
Rendimiento	Latencia por etapa y latencia total en un conjunto de prueba documentado.	Existe una línea base medible; no se fija un umbral arbitrario sin datos.
Trazabilidad	Registro de trigger solicitado, parámetros permitidos, resultado y errores.	Una auditoría puede reconstruir el flujo sin exponer secretos.

Nota: un resultado del 100 % en verde describe las suites ejecutadas, no demuestra por sí solo cobertura completa ni robustez en producción. Las métricas deben reportarse junto con el alcance de las pruebas.

4. Orden recomendado de trabajo

1. Inspeccionar estado actual

Confirmar rama main, commit de referencia, árbol limpio y comandos reales de prueba.

2. Inventariar triggers

Listar los triggers implementados y comprobar su ruta de ejecución en el backend.

3. Definir el catálogo

Establecer descripción, parámetros, permisos y estado de cada trigger.

4. Construir el selector

Añadir el desplegable junto a la barra de chat y preparar la selección en el campo de entrada.

5. Conectar con el pipeline

Garantizar que selección y escritura manual pasan por las mismas validaciones.

6. Probar escenarios límite

Validar trigger desconocido, entrada vacía, parámetros inválidos, denegación de permisos y cancelación.

7. Regresión N1–N3

Ejecutar suites automatizadas y escenarios manuales con evidencia reproducible.

8. Documentar y sincronizar

Actualizar documentación, guardar resultados, revisar diff y sincronizar el commit si las pruebas pasan.

5. Checklist de cierre de sesión

- ■ Estado inicial y commit de referencia registrados.
- ■ Catálogo de triggers verificado contra el backend.
- ■ Selector visible y utilizable desde la barra de chat.
- ■ La selección permite revisar y editar antes de enviar.
- ■ Entrada manual sigue funcionando.
- ■ No existe bypass de Nexus Root, Sovereign Gate ni de permisos.
- ■ Suites automatizadas ejecutadas y resultados guardados.
- ■ Pruebas manuales documentadas con entradas y salidas.
- ■ Deuda técnica y limitaciones registradas.
- ■ Cambios revisados, documentados y sincronizados en GitHub.

6. Criterios para avanzar más allá de alfa

El motor no debe avanzar de etapa por cantidad de funcionalidades ni por una única ejecución satisfactoria. La salida de alfa requiere una línea base reproducible, resultados de pruebas conservados, controles de seguridad verificados, fallos críticos resueltos, límites conocidos documentados y criterios de aceptación aprobados.

Decisión arquitectónica final

El selector de triggers es una capa de ergonomía y prevención de errores. El backend sigue siendo la autoridad sobre capacidades, permisos, validación y ejecución. La prioridad de HPR continúa siendo demostrar coherencia entre N1, N2 y N3 con trazabilidad y evidencia experimental.

Repositorio de referencia indicado para el proyecto: github.com/Diego5483/motor-HPR · rama main. Verificar el estado real del repositorio al comenzar la próxima sesión.